SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

삼성청년 SW·AI아카데미

AI 코어 PJT

Confidential

<알림>

본 강의는 삼성청년SW·AI아카데미의 컨텐츠로
보안서약서에 의거하여
강의 내용을 어떠한 사유로도 임의로
복사, 촬영, 녹음, 복제, 보관, 전송하거나
허가 받지 않은 저장매체를 이용한 보관,
제3자에게 누설, 공개,
또는 사용하는 등의 행위를 금합니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로! Enabling People

Day3. AI Workflow 구현

Confidential

챕터의 포인트

- API 설계
- 의도 파악과 도구 호출
- 조건 분기 처리 방식
- 대시보드 + 시뮬레이터 완성

Confidential

2회차 목표 : 멀티모달 인식 통합과 의도 파악 AI Workflow

- **오늘 : LLM 이 도구를 고르고, 그 결과로 시뮬레이터 로봇이 움직인다**
- **2일차 대시보드 서버에 도구 호출 하나를 더해 AI Workflow 를 완성한다**

차시	주제	핵심 수행 과제	산출물
1일차 (완료)	PoC 제작 + Object Detection 파인튜닝	YOLO · VLM 선정 · 테스트, PoC 서버, YOLO Finetuning	PoC 서버, best.pt
2일차 (완료)	멀티모달 인식 모델 연동	YOLO · VLM · MediaPipe 결과를 Text 로 → 웹 대시보드 서버 → LLM 통합	웹 대시보드 서버 (인식 + LLM 답변)
3일차 (오늘)	AI Workflow 구현	API 설계 → 의도 파악 + 도구 호출 → 조건 분기 → 시뮬레이터 연동	대시보드 + 시뮬레이터 동작 데모

Confidential

학습 목표

- **① 설계한다 - 브라우저와 서버가 주고받을 규격을 정하고 항목마다 번호를 매긴다**
  - 그 규격대로 만들었는지 검증 스크립트가 PASS / FAIL 로 확인하면 통과
- **② 고르게 한다 - LLM 이 도구(move / describe / ask)와 인자를 골라 JSON 으로 답한다**
  - 질문 8개에 도구를 고르게 하고, 틀린 문항이 왜 틀렸는지 설명하면 통과
- **③ 코드로 처리한다 - 조건 분기를 LLM 에 맡기지 않고 코드로 명확하게 정한다**
  - 근거 없는 이동을 되묻기로 바꾸고, 되묻기 뒤 대화를 이어가면 통과
- **④ 움직인다 - 웹캠 화면을 보고 내린 명령으로 시뮬레이터 로봇을 움직인다**
  - 대시보드에서 로봇이 움직이고, API 검증이 전부 PASS 면 통과

Confidential

3회차는 이 서버를 라즈베리파이로 옮긴다

- **① 완성된 웹 대시보드 서버**
  - /api/llm-answer 가 도구를 고른다 → 그 도구대로 시뮬레이터 로봇이 움직이고, 모르면 되묻는다
- **② API 설계 문서 + 검증 스크립트**
  - 설계대로 만들었는지 검증 스크립트가 자동으로 확인한다
- **③ 도구 호출 파서 + 조건 분기 규칙 (R1 ~ R5)**
  - LLM 이 무엇을 내놓든 형식과 규칙을 코드가 보장한다
- **④ 3회차의 출발점**
  - 라즈베리파이의 카메라 · 마이크가 같은 API 를 부르고, 도구가 실제 장치를 움직인다

Confidential

새로 설치할 것은 없다 - 2일차 환경에서 폴더만 하나 더 쓴다

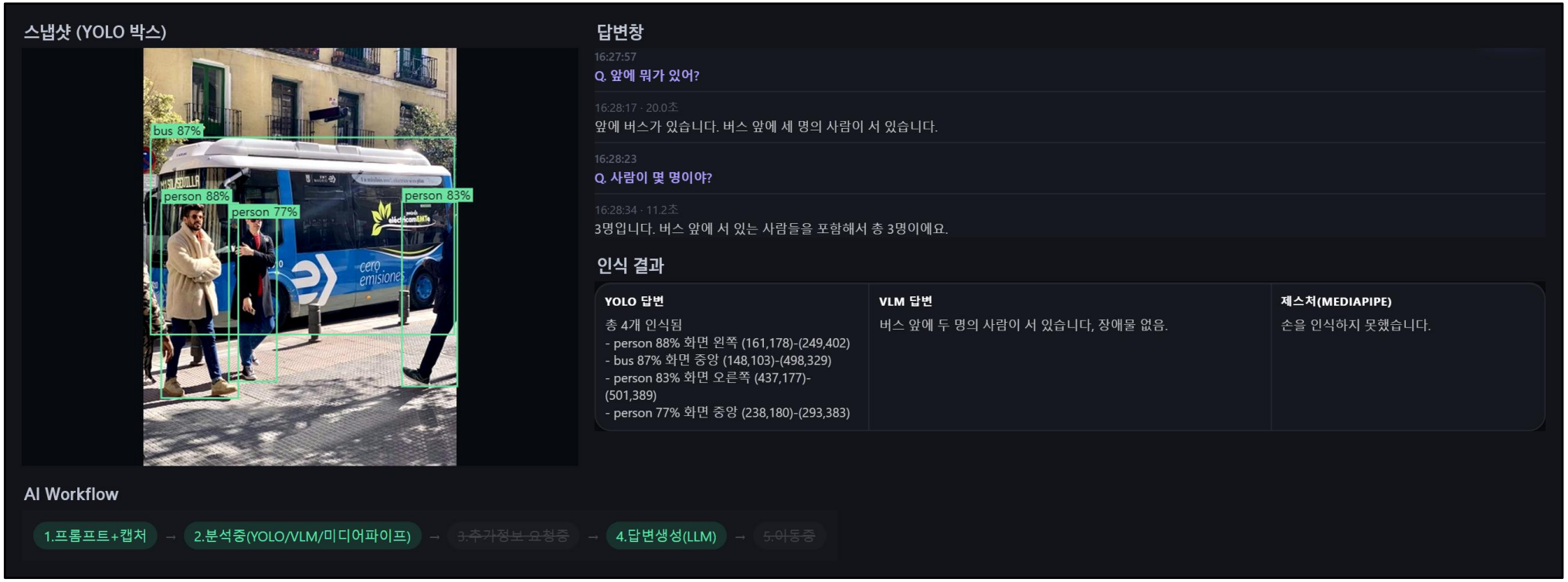
- **제스처 인식은 브라우저에서 돈다 - 파이썬 쪽 설치가 필요 없다**
- **대시보드는 서버를 띄운 뒤 http://127.0.0.1:8000 으로 연다**
- **쓰는 탭은 '대시보드 〉 로컬 PC' 와 '시뮬레이터' - '연동' 탭은 3회차용**

필요한 것	위치	확인
.venv (fastapi · ultralytics · llama_cpp · mediapipe)	core_pjt₩	2일차 page10 환경체크
가중치 5개 (YOLO · VLM · mmproj · 제스처 · LLM)	core_pjt₩models₩	2일차 page10 환경체크
대시보드 화면 (index.html + mediapipe₩)	source₩2회차_web₩	2일차 page45 에서 사용
실습 폴더 4개 + 도전 폴더 1개 + API_설계	source₩2회차_day3₩	이 강의자료의 코드 쪽번호
웹캠	(제공)	대시보드 〉 로컬 PC 에 영상이 뜬다

Confidential

설명은 하지만, 시뮬레이터상의 로봇은 움직이지 않는다

- 웹캠 질문 → YOLO · VLM → LLM 답변
- 개수는 YOLO 의 추론 결과로 처리한다(3명)
- 진행바의 '이동중' 은 건너뛴다
- 오늘 : LLM 이 도구를 고르게 한다



API 설계

Confidential

화면은 완성본 - 브라우저와 서버는 정해진 형식(API)으로만 주고받는다

- **API 하나 하나에 번호를 붙인다 (A1 ~ A5, R1 ~ R5) - API_설계와 검증 스크립트가 같은 번호를 갖는다**
- **설계 대로 만들었는지 스크립트로 PASS / FAIL 띄워 확인한다**
- **인식은 모델, 판단은 LLM, 규칙은 코드, 실행은 화면**
- **③ 판단 관련 API를 오늘 만든다**

단계	누가	무엇을	결과
① 캡처	화면	웹캠 스냅샷 1장	JPEG
② 인식	서버 A3 · A4 + 브라우저 MediaPipe	객체 · 장면 · 손 방향	Text 세 가지
③ 판단	서버 A5 (LLM + 규칙 R1 ~ R5)	사용자의 의도 → 도구	{tool, args, message}
④ 실행	화면	move → 로봇 이동 / ask → 되묻기	시뮬레이터 · 답변창

Confidential

A1 ~ A4 는 2일차 서버 그대로 - 오늘 바뀌는 것은 A5 하나

- **오류는 모두 {"error": "코드"} - 화면은 코드만 보고 안내 문구를 띄운다**

번호	경로	요청	응답	오류
A1	GET /	—	index.html	—
A2	GET /mediapipe/...	—	제스처 인식 파일	404
A3	POST /api/yolo-detect	JPEG	{objects: [{label, confidence, box}]}	400 · 500
A4	POST /api/vlm-describe	JPEG	{description}	400 · 500
A5	POST /api/llm-answer	JSON	{tool, args, need_more_info, message, user_content}	400 · 500

Confidential

LLM 에 넣을 재료는 요청에 모두 담긴다 - 서버는 저장하지 않는다

- **대화 기록(history)도 요청에 함께 담긴다 - 서버를 껐다 켜도 대화가 이어진다**

필드	예	설명
prompt (필수)	오른쪽으로 3칸 가	사용자 질문. 공백뿐이면 400 empty_prompt
yolo_summary	person 3개, bus 1개	A3 결과를 2일차 YOLO Text 규격으로 요약
vlm_description	버스 앞에 두 명의 사람이 …	A4 결과
gesture_direction	up / down / left / right / unknown	스냅샷의 손 방향 (브라우저 MediaPipe)
history	[{role, content}, …]	되묻기(ask) 뒤의 후속 턴에만 채워진다

Confidential

브라우저가 보낸 정보로 LLM 이 도구와 인자를 골라 돌려준다

- **message 는 답변창에, tool · args 는 코드로 검사한 뒤 시뮬레이터로 전달된다**
- **need_more_info = (tool == ask) - 화면의 '추가정보 요청중' 단계를 켠다**

tool	의도	args	화면 동작
move	이동 : 오른쪽으로 3칸 가	{direction, steps}	로봇을 direction 으로 steps 칸 (벽 · 경계에서 멈춤)
describe	설명 : 앞에 뭐 있어?	{}	답변만 표시. 이동 단계는 건너뜀
ask	방향 모름 : 저쪽으로 가	{} 또는 {steps}	되묻고, 대화 기록을 다음 전송에 함께 보냄

Confidential

LLM 은 제안하고, 코드가 확정한다

- **R1 · R2 는 파서(의도 파악과 도구 호출), R3 · R5 는 조건 분기에서 만든다**

번호	규칙	LLM 의 제안 → 코드의 확정	만드는 곳
R1	tool 은 항상 move · describe · ask 중 하나 (깨진 JSON · 없는 도구 → describe)	깨진 JSON → describe	실습 2-1
R2	도구별 인자 형식 — 방향은 네 개 중 하나, 칸 수 1 ~ 10. 방향 없는 move → ask	방향 없는 move(steps 3) → ask	실습 2-1
R3	방향의 근거가 입력에 없으면 move 를 ask 로 바꾼다 (LLM 이 고른 ask 도 같은 형식)	'저쪽으로 가' 에 move(left, 1) → ask	실습 3-1
R4	need_more_info 는 tool == ask 와 같다	ask → need_more_info true	실습 4-1
R5	ask 뒤에 방향만 답하면 move — 칸 수는 ask 가 기억한 값, 방향은 질문의 방향어가 우선	ask 뒤 '왼쪽' 에 또 ask → move(left, 2)	실습 3-1

Confidential

번호마다 요청을 보내고 PASS / FAIL 을 찍는다

- **기대 도구와 인자를 표로 적는다**

  예 : '오른쪽으로 3칸 가' → move {direction: right, steps: 3}

- **모든 응답에 형식 검사가 붙는다**
- **서버를 먼저 띄운다 (터미널 1)**

  cd ..₩2회차_day2₩page64_대시보드_LLM연결
  python 대시보드_LLM연결.py

- **실행 : python API_검증.py**

```
    ("A5-2", "방향 · 칸 수 → move", "오른쪽으로 3칸 가", "unknown", "move",
     {"direction": "right", "steps": 3}),

# ...

    for item_id, title, prompt, gesture, tool, args in TOOL_CASES:
        r, sec = llm(client, prompt, gesture)
        data = r.json() if r.status_code == 200 else {}
        got = f"{data.get('tool')} {data.get('args', '')}"
        check(item_id, f"{title} : {prompt!r}",
              data.get("tool") == tool and data.get("args") == args, f"{sec:.1f}s → {got}")
        if data:
            check_shape(data)
```

전체 소스 : page16_API_검증/API_검증.py
https://gist.github.com/maketrust7/5595df07495591c8656845f6a8ec3aa9

Confidential

17 / 43 - 무엇이 비었는지 번호로 보인다

- **A1 ~ A4 는 모두 PASS**
- **A5 · R1 ~ R3 · R5 FAIL - 응답에 tool 이 없다**
- **오늘 할 일 = FAIL 을 PASS 로**
- **끝나면 2일차 서버를 끈다 (Ctrl+C)**

명령 프롬프트

```
[A3 · A4] 인식 (sample.jpg 810x1080)
PASS  [A3  ] objects : label · confidence · box(픽셀 좌표)  — 4.0s  bus, person, person, person,
      person
  PASS  [A4  ] description 문자열  — 10.4s  '버스 앞에 사람들이 걷고 있다, 장애물 없음.'

[A5 · R3] 의도 파악 → 도구 선택
  FAIL  [A5-1] 장면 질문 → describe : '앞에 뭐가 있어?'  — 6.8s → None
  FAIL  [A5-2] 방향 · 칸 수 → move : '오른쪽으로 3칸 가'  — 2.9s → None
  FAIL  [A5-3] 방향은 손 제스처 → move : '이쪽으로 2칸 이동해'  — 2.5s → None
  FAIL  [A5-4] 제스처 시퀀스 입력 문구 → move : '왼쪽으로, 5칸 이동.'  — 3.3s → None
  FAIL  [R3  ] 방향 근거 없음 → ask : '저쪽으로 가'  — 2.5s → None
  FAIL  [R3  ] 방향 근거 없음 → ask : '버스 있는 쪽으로 가'  — 2.2s → None

[R5] 되묻기 → 방향을 답하면 이어서 이동
  FAIL  [R5  ] 1턴 '저쪽으로 2칸 가' → ask  — → None
  FAIL  [R5  ] 2턴 '왼쪽' → move(left, 2)  — 3.3s → None

------------------------------------------------------------
A1 1/1  A2 2/2  A3 3/3  A4 3/3  A5 2/12  R1 0/6  R2 0/6  R3 0/2  R4 6/6  R5 0/2
전체 17/43 PASS  — FAIL 번호를 ../API_설계 에서 찾는다
```

의도 파악과 도구 호출

Confidential

사용자의 프롬프트를 '호출할 함수 하나' 로 바꾼다

- **사용자의 말은 세 종류로 나뉜다**
  - '앞에 뭐 있어?' → 설명, '오른쪽으로 3칸 가' → 이동, '저쪽으로 가' → 이동인데 방향을 모름
- **종류마다 도구가 하나 - LLM 이 이 중 하나를 고른다**
  - describe() : 설명만 한다 (YOLO · VLM 결과는 고르기 전에 이미 받아 둔 입력)
  - move(direction, steps) : 로봇을 direction 으로 steps 칸 움직인다
  - ask() : 방향을 몰라 되묻는다
- **LLM 은 로봇을 직접 움직이지 않는다**
  - 고른 도구와 인자를 JSON 으로 돌려줄 뿐 - 그 도구대로 실행되고, 코드로 검사한다
  - 그래서 LLM 이 틀려도 되돌릴 기회가 있다 (조건 분기)

Confidential

LLM 에게 도구를 알려 주는 곳 = 시스템 프롬프트

- 도구마다 한 줄 : 이름 · 인자 · 언제
- 근거 순서와 '모르면 ask' 도 적는다
- 출력은 JSON 한 개 {tool, args, message}
- 파서 R1 : 깨진 JSON · 없는 도구 → describe
- 파서 R2 : '오른쪽' → right, 칸 1 ~ 10
- 방향 없는 move → ask

```
SYSTEM_PROMPT = (
    "너는 격자 시뮬레이터의 로봇을 조종하는 어시스턴트다. 인식 모델이 만든 Text 블록과 "
    "사용자 질문을 보고 사용자의 의도를 파악해 도구 하나를 고른다.\n"
    "도구:\n"
    "- move(direction, steps): 로봇 이동. direction 은 up|down|left|right, steps 는 1~10 정수\n"
    "- describe(): 이동 없이 장면에 대한 질문에 답한다\n"
    "- ask(): 이동하라는데 방향을 알 수 없을 때 사용자에게 되묻는다\n"
    "규칙:\n"
    "- 방향은 [사용자 질문]에 있으면 그것을, 없으면 [손 제스처]를 따른다. "
    "둘 다 없으면 move 가 아니라 ask 를 고른다.\n"
    "- 칸 수가 없으면 1 이다.\n"
    "- 사람 · 물체의 개수는 반드시 [YOLO 객체 인식] 의 숫자를 쓴다. "
    "[VLM 장면 설명] 에 나온 개수는 쓰지 않는다.\n"
    "- 상황과 분위기는 [VLM 장면 설명] 을 참고한다.\n"
    "반드시 아래 JSON 한 개로만 답하고, 다른 글은 쓰지 마라. "
    "args 는 move 일 때만 채우고 나머지 도구는 {} 로 둔다.\n"
    '{"tool": "move|describe|ask", "args": {"direction": "up", "steps": 1}, '
    '"message": "<사용자에게 보일 한두 문장>"}'
```

Confidential

LLM 이 고르고, 파서가 형식을 보장한다

- **인식 결과는 2일차 대시보드에서 받은 값**
- **질문 8개의 기대 도구와 비교**
- **실행 : python 도구선택.py**

```python
def parse_tool_call(raw: str) -> dict:
    data = extract_json(raw)
    if data is None:                        # JSON 이 아니면 설명으로 본다
        return {"tool": "describe", "args": {}, "message": raw or FALLBACK_MESSAGE}
    tool = str(data.get("tool", "")).strip().lower()
    if tool not in TOOLS:                   # 없는 도구 → describe
        tool = "describe"
    message = str(data.get("message", "")).strip()
    args = {}
    if tool == "move":
        raw_args = data["args"] if isinstance(data.get("args"), dict) else data
        direction = to_direction(raw_args.get("direction"))
        if direction == "unknown":          # 방향 없는 move → ask
            tool = "ask"
        else:
            args = {"direction": direction,
                    "steps": to_steps(raw_args.get("steps"))}
    if tool == "ask" and not message:
        message = ASK_MESSAGE
    return {"tool": tool, "args": args, "message": message or FALLBACK_MESSAGE}
```

전체 소스 : page21_도구선택/도구선택.py
https://gist.github.com/maketrust7/174390615f0048b59d595e9238b367d3

Confidential

모르면 되묻지 않고 '오른쪽' 으로 추측한다

- 설명 2 · 이동 4 문항은 모두 정답
- 손 제스처 '위' 도 방향으로 썼다
- '저쪽' · '버스 있는 쪽' → move(right, 1)
- 시스템 프롬프트에 '모르면 되묻기(ask)' 가 있는데도 LLM 이 move() 를 골랐다

```
SYSTEM_PROMPT = (
    "너는 격자 시뮬레이터의 로봇을 조종하는 어시스턴트다. 인식 모델이 만든 Text 블록과 "
    "사용자 질문을 보고 사용자의 의도를 파악해 도구 하나를 고른다.\n"
    "도구:\n"
    "- move(direction, steps): 로봇 이동. direction 은 up|down|left|right, steps 는 1~10 정수\n"
    "- describe(): 이동 없이 장면에 대한 질문에 답한다\n"
    "- ask(): 이동하라는데 방향을 알 수 없을 때 사용자에게 되묻는다\n"
    "규칙:\n"
    "- 방향은 [사용자 질문]에 있으면 그것을, 없으면 [손 제스처]를 따른다. "
    "둘 다 없으면 move 가 아니라 ask 를 고른다.\n"
    "- 칸 수가 없으면 1 이다.\n"
    "- 사람 · 물체의 개수는 반드시 [YOLO 객체 인식] 의 숫자를 쓴다. "
    "[VLM 장면 설명] 에 나온 개수는 쓰지 않는다.\n"
    "- 상황과 분위기는 [VLM 장면 설명] 을 참고한다.\n"
    "반드시 아래 JSON 한 개로만 답하고, 다른 글은 쓰지 마라. "
    "args 는 move 일 때만 채우고 나머지 도구는 {} 로 둔다.\n"
    '{"tool": "move|describe|ask", "args": {"direction": "up", "steps": 1}, '
    '"message": "<사용자에게 보일 한두 문장>"}'
```

명령 프롬프트

```
모델 로딩 중...
[인식 결과] YOLO: person 3개, bus 1개 / VLM: 버스 앞에 두 명의 사람이 서 있습니다, 장애물 없음.

O  6.3s  앞에 뭐가 있어?
      → describe()  "앞에 사람 2명이 서 있습니다."
O  1.7s  사람이 몇 명이야?
      → describe()  "3명입니다."
O  2.9s  오른쪽으로 3칸 가
      → move(right, 3)  "오른쪽으로 3칸 이동합니다."
O  2.9s  아래로 한 칸만 가줘
      → move(down, 1)  "아래로 한 칸 이동합니다."
O  2.9s  왼쪽으로, 5칸 이동.
      → move(left, 5)  "왼쪽으로 5칸 이동합니다."
O  2.9s  이쪽으로 2칸 이동해 [손: 위]
      → move(up, 2)  "위로 2칸 이동합니다."
X  2.8s  저쪽으로 가
      → move(right, 1)  "오른쪽으로 가겠습니다."   (기대 ask())
X  2.9s  버스 있는 쪽으로 가
      → move(right, 1)  "버스 쪽으로 가고 있습니다."   (기대 ask())

정답 6/8
```

Confidential

잘못된 추측은 틀린 설명보다 위험하다

- **모델은 일단 행동하는 쪽을 고르는 경향이 있다**
  - 이동 요청처럼 보이면 move 를 고르고, 비어 있는 방향은 그럴듯한 값으로 채운다
- **4B 모델은 조건이 겹친 규칙을 잘 지키지 못한다**
  - '질문에도 없고 손에도 없으면 ask' - 두 조건을 함께 확인해야 한다
- **틀린 이동은 되돌리기 어렵다**
  - 설명이 틀리면 다시 물으면 되지만, 로봇은 엉뚱한 쪽으로 이미 움직인 뒤다

도전 문제 1 - 프롬프트만으로 되묻게 할 수 있을까

Confidential

--rule 로 시스템 프롬프트에 규칙 한 줄을 더해, '저쪽으로 가' 에 LLM 이 되묻게(ask()) 하시오.

먼저 종이에 적고 시작하세요

Q1. 규칙 한 줄로 8/8 이 될까?
Q2. 된다면, 질문을 바꿔도 그럴까?
Q3. 안 된다면, 어디서 막아야 할까?

실행

python 도구선택.py --rule "..."

규칙 예 :
'저쪽', '이쪽' 처럼 가리키는 말만
있고 손 제스처를 인식하지 못하면
반드시 ask 를 고른다

Confidential

프롬프트는 '부탁' 일 뿐이다 - 지킬지는 모델에 달렸다

- **규칙 1 : 가리키는 말 + 손 없음 → ask (조건을 나열)**
- **규칙 2 : '추측하지 마라' + 방향어 목록 (금지 + 조건)**
- **둘 다 같은 두 문항에서 move(right, 1) 을 골랐다 → 판단을 코드로 옮긴다 (조건 분기)**

규칙 두 가지로 실행 (실측, 틀린 문항만)

```
[규칙 1] '저쪽', '이쪽', '그쪽' 처럼 가리키는 말만 있고 [손 제스처] 가 인식 불가면 반드시 ask 를 고른다.
  X  저쪽으로 가  →  move(right, 1)  "오른쪽으로 가겠습니다."
  X  버스 있는 쪽으로 가  →  move(right, 1)  "버스 쪽으로 가고 있습니다."
  정답 6/8  (나머지 6문항은 그대로 O)

[규칙 2] 방향을 추측하지 마라. 질문에 오른쪽·왼쪽·위·아래가 없고 [손 제스처] 도 인식 불가면 ask 를 고른다.
  X  저쪽으로 가  →  move(right, 1)  "오른쪽으로 가겠습니다."
  X  버스 있는 쪽으로 가  →  move(right, 1)  "버스 방향으로 가고 있습니다."
  정답 6/8  (나머지 6문항은 그대로 O)
```

Confidential

API(주고받는 형식)를 정했고, LLM 이 도구를 고르게 했다

- ① 브라우저와 서버는 정해진 형식(API)으로만 주고받는다 - API(A1 ~ A5)와 규칙(R1 ~ R5) 번호로 설계와 검증을 맞춘다
- ② 2일차 서버는 응답에 tool 이 없어서 A5 · R1 ~ R3 · R5 가 FAIL 이었다
- ③ 의도 파악 = 도구 고르기 - LLM 은 describe · move · ask 중 하나를 고르고, 파서가 형식을 보장한다 (R1 · R2)
- ④ LLM 은 로봇을 직접 움직이지 않는다 - message 는 답변창에, tool · args 는 코드로 검사한 뒤 시뮬레이터로 전달된다
- ⑤ LLM 은 방향을 모르면 되묻지 않고 추측했다. 시스템 프롬프트에 규칙을 더해도 마찬가지였다
- ⑥ 다음 : 판단을 코드로 옮긴다 (조건 분기) - 근거 없는 move 는 ask 로 바꾼다

조건 분기 처리 방식

Confidential

형식뿐 아니라 판단 조건도 코드로 지킨다

- **순서 : LLM → 파서 (R1 · R2) → 규칙 (R3 · R5) → 응답**
- **R3 - 근거 없는 move 는 ask 로 바꾼다**
  - LLM 이 move 를 골라도 입력에 방향의 근거가 없으면 ask 로 바꿔 되묻는다
- **근거로 인정하는 것은 입력에 실제로 있는 정보뿐이다**
  - 질문의 방향어 · 이전 턴의 질문 · 손 제스처 방향
  - LLM 이 채운 direction 은 근거가 아니라 추측이다
- **2일차의 '형식은 파서가 보장한다' 에서 한 걸음 더 나아간다**

Confidential

입력에 적혀 있는 것만 근거로 쓴다

- **'버스 있는 쪽' 으로 가려면 버스 위치를 알아야 하는데, 지금 입력에는 그 정보가 없다**

입력	예	근거?
질문의 방향어	오른쪽으로 3칸 가	O
손 제스처	이쪽으로 2칸 이동해 + 손 '위'	O
되묻기 뒤의 답	'저쪽으로 2칸 가' → (ask) → '왼쪽'	O
가리키는 말	저쪽으로 가	X → ask
장면 속 물체	버스 있는 쪽으로 가	X → ask

Confidential

대화 기록은 브라우저가 보관하고, 칸 수는 코드가 이어받는다

- **ask 뒤에도 대화는 이어진다 (R5)**
  - 브라우저가 이번 턴의 입력과 답을 대화 기록(history)에 담아 두었다가 다음 요청에 함께 보낸다
- **칸 수 기억은 LLM 에게 맡기지 않는다**
  - '저쪽으로 2칸 가' 다음에 '왼쪽' 이라고 답하자 LLM 은 칸 수를 1칸으로 바꿨다
  - R3 가 ask 로 바꿀 때 이미 아는 칸 수를 args 에 담아 두고, 다음 턴에 코드가 그 값을 이어서 쓴다
  - '아래' 처럼 방향만 답했는데 LLM 이 또 ask 를 골라도 코드가 move 로 바꾼다
- **되묻는 문장에도 이미 아는 정보를 넣는다**
  - '어느 쪽으로 2칸 갈까요?' - 사용자는 방향만 답하면 된다는 것을 바로 안다

Confidential

LLM 이 고른 것 → 코드가 확인한 것

- **시스템 프롬프트는 실습 2-1 과 같다**
- **근거가 없으면 ask (R3)**
- **되묻기 뒤 칸 수는 코드가 이어받는다 (R5)**
- **실행 : python 조건분기.py**

```python
def has_direction_evidence(prompt: str, history: list, gesture_direction: str) -> bool:
    if gesture_direction in DIRECTIONS:
        return True
    questions = [prompt] + [m.group(1) for turn in history if turn["role"] == "user"
                            for m in [_QUESTION_RE.search(turn["content"])] if m]
    return any(_DIRECTION_WORD_RE.search(q) for q in questions)


# ...

def apply_rules(call: dict, prompt: str, history: list, gesture_direction: str) -> dict:
    said = said_direction(prompt)
    steps = said_steps(prompt)
    pending = pending_steps(history)
    if call["tool"] == "move":
        if not has_direction_evidence(prompt, history, gesture_direction):
            return ask_for_direction(steps or call["args"]["steps"])       # [R3] 근거 없는 이동
        return move_with_steps(said or call["args"]["direction"],          # [R3] 적힌 방향이 먼저
                               steps or pending or call["args"]["steps"])  # [R5] 칸 수: 질문 → 기억 → LLM
    if call["tool"] == "ask":
        if pending and said:
            return move_with_steps(said, steps or pending)                 # [R5] 방향만 답해도 이동
        return ask_for_direction(steps or pending or 1)                    # [R3] 칸 수를 기억해 되묻기
    return call
```

전체 소스 : page31_조건분기/조건분기.py
https://gist.github.com/maketrust7/fa6cb640187c9c704c7408140b15cfff

Confidential

8 / 8 - R3 가 두 문항을 ask 로 바꿨고, 되묻기는 2턴에 끝났다

- **LLM 은 여전히 move(right, 1) 을 골랐다**
- **R3 가 ask(steps=1) 로 바꿨다**
- **2턴 '왼쪽' → move(left, 2)**
- **칸 수 2 는 코드가 이어받았다**

명령 프롬프트

```
모델 로딩 중...

[1] 채점 — LLM 이 고른 것 → 규칙 적용 후
O 앞에 뭐가 있어?
      LLM describe()       → 최종 describe()
O 사람이 몇 명이야?
      LLM describe()       → 최종 describe()
O 오른쪽으로 3칸 가
      LLM move(right, 3)   → 최종 move(right, 3)
O 아래로 한 칸만 가줘
      LLM move(down, 1)    → 최종 move(down, 1)
O 왼쪽으로, 5칸 이동.
      LLM move(left, 5)    → 최종 move(left, 5)
O 이쪽으로 2칸 이동해 [손: 위]
      LLM move(up, 2)      → 최종 move(up, 2)
O 저쪽으로 가
      LLM move(right, 1)   → 최종 ask(steps=1)  ← 코드가 바꿈
O 버스 있는 쪽으로 가
      LLM move(right, 1)   → 최종 ask(steps=1)  ← 코드가 바꿈

정답 8/8

[2] 되묻기 → 방향을 답하면 이어서 (history)
  1턴  Q. 저쪽으로 2칸 가    → ask(steps=2)  "어느 쪽으로 2칸 갈까요? 방향을 말하거나 "
  2턴  Q. 왼쪽           → move(left, 2)  "왼쪽으로 2칸 이동합니다."
  대화 기록 4개 → 방향은 2턴째 답에서, 칸 수는 1턴째 ask 가 기억한 값에서
```

Confidential

'위 2칸 가' 는 방향이 분명하니 move(up, 2) 가 되어야 하는데 ask 로 바뀐다. 조건분기_도전2.py 의 방향어 검사를 고치시오.

지금 - 왜 ask 가 되나

python 조건분기_도전2.py

X 위 2칸 가

LLM move(up, 2) → 최종 ask(steps=2)

← R3 가 move 를 ask 로 바꿨다

R3 (15쪽) : 방향의 근거가 없으면 move → ask

원인 : '위' 를 방향어로 못 찾아 근거가 없다고 봤다

요구 사항과 힌트

목표 : 방향이 있으면 move, 없으면 ask

① '위 2칸 가' → move(up, 2)

② '버스 위치로 가' → 여전히 ask

(버스가 어느 쪽인지 입력에 없다)

③ 실습 3-1 의 질문 8개(32쪽)도 계속 정답

· DIRECTION_WORD_RE 한 줄만 고친다

· '위치' 에도 '위' 가 들어 있다

- 방향의 '위' 뒤에는 무엇이 오나?

전체 소스 : page33_조건분기/조건분기_도전2.py

https://gist.github.com/maketrust7/75afaea777601d80eb45d2bb0911a772

Confidential

규칙을 고친 뒤에는 막아야 할 것과 통과시킬 것을 둘 다 확인한다

- '위쪽|위로' 만 찾으면 '위' 를 놓친다
- '위' 만 넣으면 '위치' 도 방향이 된다
- '위 '(위 + 공백) · 문장 끝 '위' 를 더한다
- 수정 후 세 문항 모두 O
- 실습 3-1 질문 8개도 계속 정답

```python
# 수정 전 (조건분기_도전2.py) — '위쪽', '위로' 만 방향어로 본다
DIRECTION_WORD_RE = re.compile(
    r"오른쪽|왼쪽|위쪽|위로|아래|\b(?:up|down|left|right)\b", re.I)

# 수정 후 (조건분기_도전2_정답.py)
# 방향어 목록 — 위 : 위쪽 · 위로 · '위 '(위 + 공백) · 문장 끝의 위($),  아래 : 아래쪽 · 아래로 포함
# ('위치', '위에' 는 방향이 아니다)
DIRECTION_WORD_RE = re.compile(
    r"오른쪽|오른편|우측|왼쪽|왼편|좌측|아래|밑|위쪽|위로|위 |위$|\b(?:up|down|left|right)\b", re.I)
```

검증 — 수정 전 / 후 실제 실행

```
[수정 전] 조건분기_도전2.py — 위쪽|위로 만 방향어
  X  위 2칸 가       LLM move(up, 2)       → 최종 ask(steps=2)
  O  버스 위치로 가    LLM move(right, 1)    → 최종 ask(steps=1)
  O  위에 뭐 있어?    LLM describe()        → 최종 describe()
  기존 8문항 : 8/8

[수정 후] 조건분기_도전2_정답.py — '위 ' · 문장 끝 '위' · 동의어(우측 · 밑 …) 추가
  O  위 2칸 가       LLM move(up, 2)       → 최종 move(up, 2)
  O  버스 위치로 가    LLM move(right, 1)    → 최종 ask(steps=1)
  O  위에 뭐 있어?    LLM describe()        → 최종 describe()
  기존 8문항 : 8/8
```

대시보드 + 시뮬레이터 완성

Confidential

/api/llm-answer 의 출력이 '설명 문장' 에서 '도구 호출' 로 바뀐다

- **A1 ~ A4 는 그대로**
  - 화면 · 제스처 파일 · YOLO · VLM 은 2일차 서버(실습 4-3) 코드를 그대로 쓴다
- **A5 : decide_tool = 실습 2-1 의 파서 + 실습 3-1 의 규칙 (apply_rules)**
  - LLM 입력 블록([YOLO 객체 인식] …)은 2일차와 같다
- **화면(index.html)은 한 줄도 고치지 않는다**
  - 화면은 처음부터 tool == move 일 때만 시뮬레이터 로봇이 움직이도록 만들어져 있다
  - 응답에 tool · args 가 담기는 순간부터 시뮬레이터 로봇이 움직인다

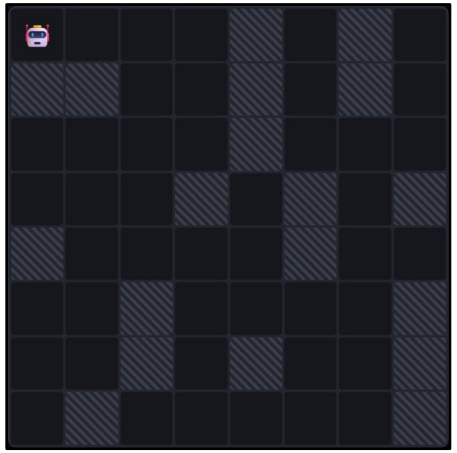
번호	경로	요청	응답	오류
A1	GET /	–	index.html	–
A2	GET /mediapipe/...	–	제스처 인식 파일	404
A3	POST /api/yolo-detect	JPEG	{objects: [{label, confidence, box}]}	400 · 500
A4	POST /api/vlm-describe	JPEG	{description}	400 · 500
A5	POST /api/llm-answer	JSON	{tool, args, need_more_info, message, user_content}	400 · 500

Confidential

로봇의 위치와 벽 정보는 브라우저에만 있다

- **8x8 격자, 출발점은 (1, 1) - 왼쪽 위**
  - 좌표는 (행, 열). up 은 행 −1, down 은 행 +1, right 는 열 +1
- **벽 19칸은 페이지를 열 때 무작위로 생긴다**
  - 벽을 뽑은 뒤 (1, 1)에서 BFS 로 모든 빈칸에 닿는지 검사한다 - 갇힌 칸이 생기면 다시 뽑는다 (index.html 에 포함)
- **move 를 받으면 로봇은 한 칸씩 가다가 벽이나 경계를 만나면 멈춘다**
  - 멈춘 사실은 이동 경로 로그에만 남고 서버에는 전달되지 않는다

```
function isFullyConnectedFromStart(wallMap, openCount) {
  // ... 시작 칸 (1, 1) 을 visited · queue 에 넣는다
  while (queue.length) {
    var current = keyToRowCol(queue.shift());
    // 상 · 하 · 좌 · 우 이웃 칸
    [[current.row - 1, current.col], [current.row + 1, current.col],
     [current.row, current.col - 1], [current.row, current.col + 1]].forEach(function (pair) {
      // ... 격자 밖이면 건너뛴다
      if (wallMap[key] || visited[key]) return;
      visited[key] = true;
      queue.push(key);
    });
  }
  return Object.keys(visited).length === openCount;   // 방문 칸 수 == 빈칸 수
}
```



도구가 실행되는 곳 - index.html (읽기만)

Confidential

브라우저는 응답의 tool 대로 실행하고, 서버는 형식(API)만 지킨다

- move 일 때만 시뮬레이터 로봇이 움직인다
- describe(설명만) 면 '이동중' 을 건너뛴다
- ask 면 대화 기록을 보관하고 대기한다
- 한 칸씩 가다가 벽 · 경계에서 멈춘다
- 결과는 로그에만 남는다 (서버로 안 보냄)

```
    if (llmResult.tool !== 'move') {
      steps[WORKFLOW_MOVE_STEP_INDEX].classList.add('skipped');
      return;
    }
    var moveDirection = llmResult.args.direction;
    var moveSteps = Number(llmResult.args.steps);

    if (robotId === LOCAL_PC_ROBOT_ID) applyRobotMove(moveDirection, moveSteps);

// ...

  function applyRobotMove(direction, steps) {
    var stepCount = Number.isFinite(steps) && steps > 0 ? Math.floor(steps) : 1;
    var prevRow = robot.row, prevCol = robot.col;

    var blockedByWall = false;
    var movedCells = 0;
    if (direction === 'up' || direction === 'down' || direction === 'left' || direction === 'right') {
      robot.facing = direction;
      var delta = DIRECTION_DELTA[direction];
      for (var i = 0; i < stepCount; i++) {
        var nextRow = robot.row + delta.row;
        var nextCol = robot.col + delta.col;
        var inBounds = nextRow >= 1 && nextRow <= GRID_SIZE && nextCol >= 1 && nextCol <= GRID_SIZE;
        var isWall = inBounds && !!walls[nextRow + '-' + nextCol];
        if (!inBounds || isWall) {
          if (isWall) blockedByWall = true;
          break; // 더 이상 진행할 수 없으면 그 자리에서 멈춘다.
        }
        robot.row = nextRow;
        robot.col = nextCol;
        movedCells++;
```

Confidential

2일차 실습 4-3 서버 + decide_tool

- 응답에 tool · args · need_more_info 가 붙는다
- 요청마다 서버 창에 [tool] 로그 한 줄
- 실행 : python 대시보드_완성.py
- 접속 : http://127.0.0.1:8000

```
        call, user_content = await run_in_threadpool(
            decide_tool,
            prompt,
            str(body.get("yolo_summary", "")),
            str(body.get("vlm_description", "")),
            to_direction(body.get("gesture_direction", "unknown")),
            _read_history(body.get("history")))

# ...

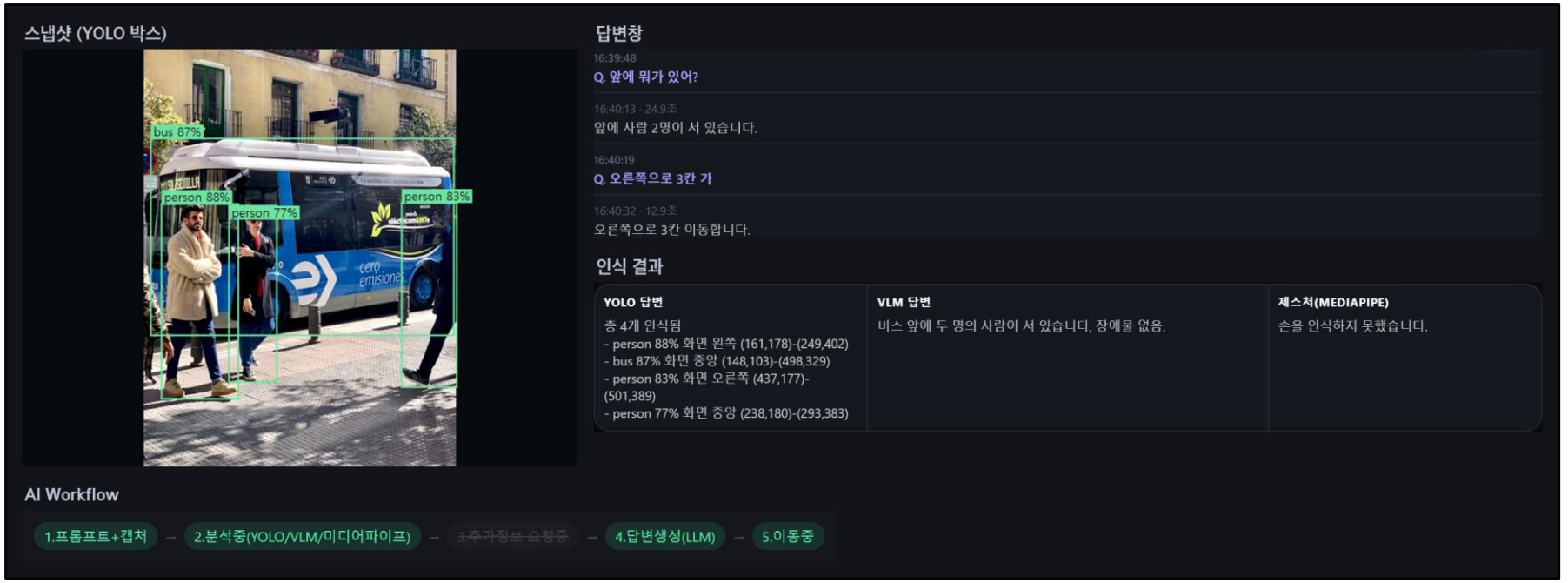
    return {**call,
            "need_more_info": call["tool"] == "ask",     # [R4]
            "user_content": user_content}
```

전체 소스 : page39_대시보드_완성/대시보드_완성.py
https://gist.github.com/maketrust7/8c2c80004c440537e023ecfd644d3462

Confidential

질문에 따라 도구가 달라진다 - '이동중' 단계가 처음 켜졌다

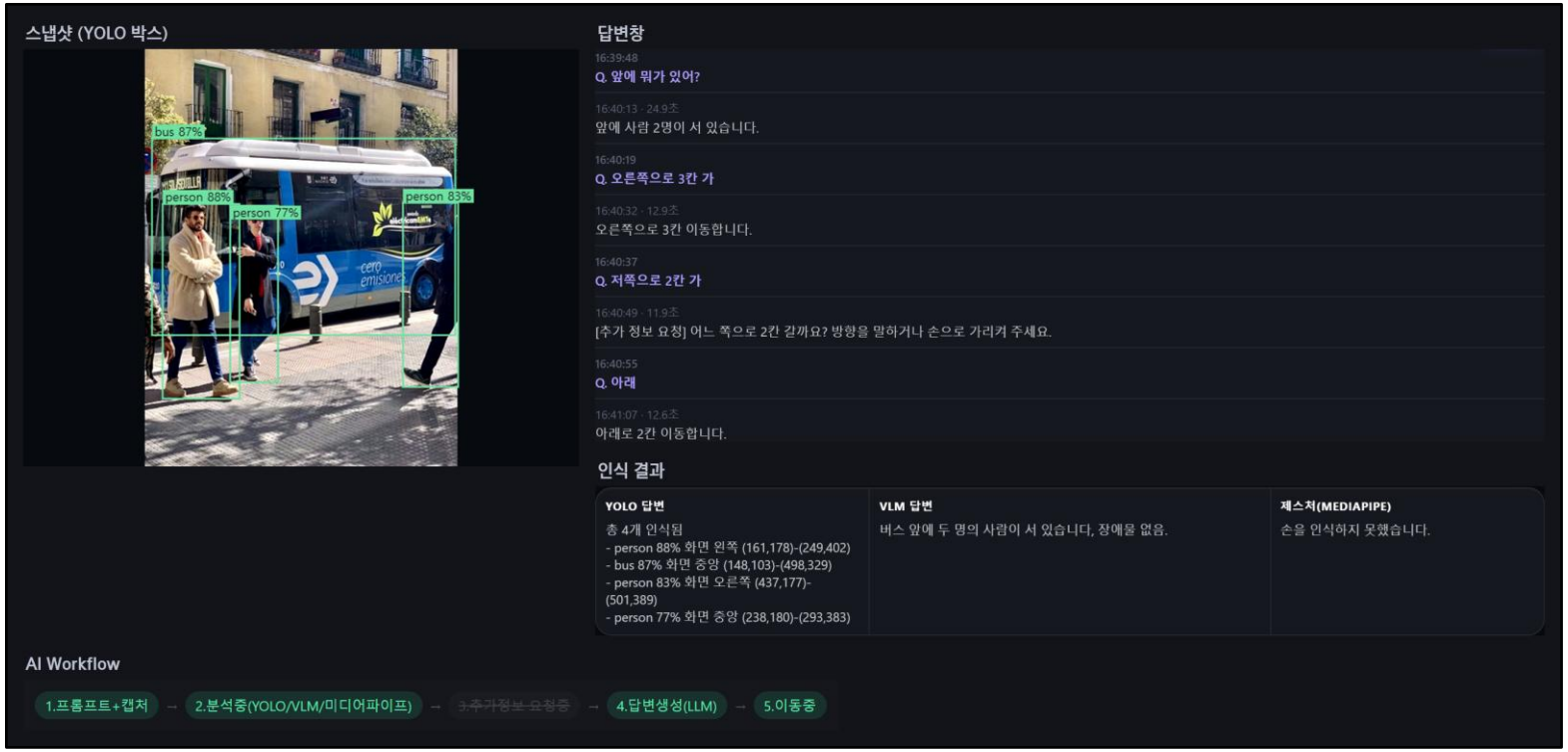
- **Q. 앞에 뭐가 있어? → describe**
- **Q. 오른쪽으로 3칸 가 → move(right, 3)**
- **진행바가 '5.이동중' 까지 켜졌다**



Confidential

방향만 묻고, 칸 수는 기억한다

- **Q. 저쪽으로 2칸 가 → [추가 정보 요청]**
- **'어느 쪽으로 2칸 갈까요?'**
- **Q. 아래 → 아래로 2칸 이동**
- **대화 기록은 브라우저가 보관한다**



Confidential

(1, 1) → 오른쪽 3칸 → 아래 2칸

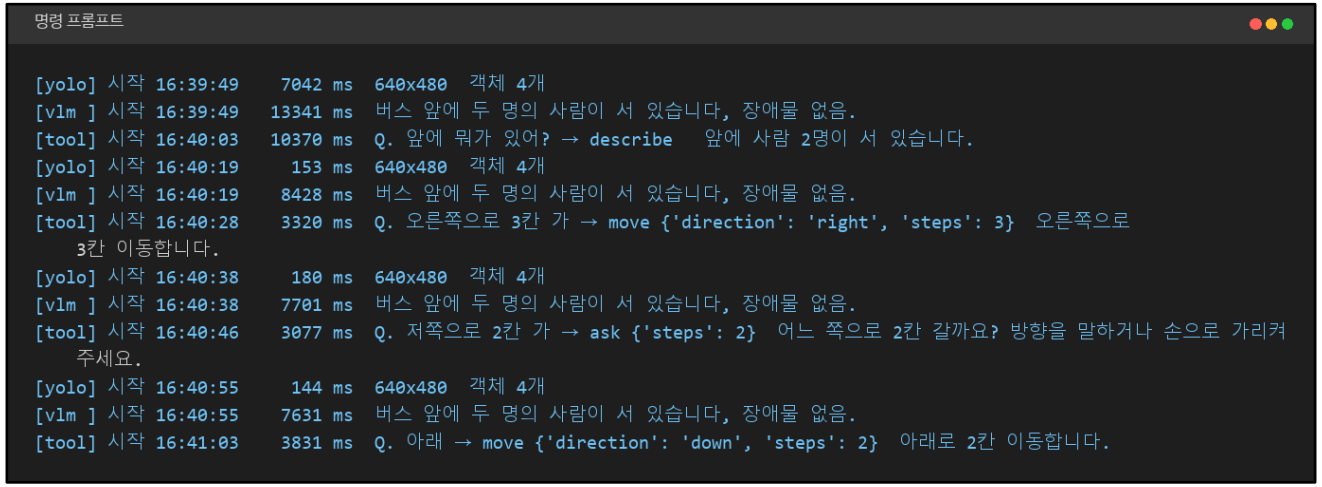
- **오른쪽 3칸 : (1, 1) → (1, 4)**
- **아래 2칸 : (1, 4) → (3, 4)**
- **지나온 경로가 표시된다**



Confidential

[tool] 로그 한 줄에 LLM 의 판단이 모두 보인다

- **질문 → 도구 → 인자 → 문장**
- **LLM 판단 3.1 ~ 3.8 s**
- **첫 요청 10.4 s (모델 로딩)**
- **[전송] 한 번에 약 12초 (VLM + LLM)**



실습 4-2 - 동작 결과 (완성 서버)

Confidential

설계대로 만들어 졌는지 확인한다

- 2일차 서버 17/43 → 오늘 43/43
- A5 · R1 ~ R5 모두 PASS
- 브라우저 없이 스크립트로 확인한다

```
명령 프롬프트

[A3 · A4] 인식 (sample.jpg 810x1080)
PASS  [A3  ] objects : label · confidence · box(픽셀 좌표)  — 3.9s  bus, person, person, person,
      person
  PASS  [A4  ] description 문자열  — 11.1s  '버스 앞에 사람들이 걷고 있다, 장애물 없음.'

[A5 · R3] 의도 파악 → 도구 선택
  PASS  [A5-1] 장면 질문 → describe : '앞에 뭐가 있어?'  — 9.8s → describe {}
  PASS  [A5-2] 방향 · 칸 수 → move : '오른쪽으로 3칸 가'  — 3.2s → move {'direction': 'right', 'steps': 3}
  PASS  [A5-3] 방향은 손 제스처 → move : '이쪽으로 2칸 이동해'  — 3.2s → move {'direction': 'up', 'steps': 2}
PASS  [A5-4] 제스처 시퀀스 입력 문구 → move : '왼쪽으로, 5칸 이동.'  — 3.4s → move {'direction': 'left',
      'steps': 5}
  PASS  [R3  ] 방향 근거 없음 → ask : '저쪽으로 가'  — 2.9s → ask {'steps': 1}
  PASS  [R3  ] 방향 근거 없음 → ask : '버스 있는 쪽으로 가'  — 3.2s → ask {'steps': 1}

[R5] 되묻기 → 방향을 답하면 이어서 이동
  PASS  [R5  ] 1턴 '저쪽으로 2칸 가' → ask  — → ask {'steps': 2}
  PASS  [R5  ] 2턴 '왼쪽' → move(left, 2)  — 5.5s → move {'direction': 'left', 'steps': 2}

------------------------------------------------------------
A1 1/1  A2 2/2  A3 3/3  A4 3/3  A5 12/12  R1 6/6  R2 6/6  R3 2/2  R4 6/6  R5 2/2
전체 43/43 PASS
```

Confidential

답은 '3칸 이동', 실제로는 1칸 - 서버는 실제 이동 결과를 받지 않는다

- '오른쪽으로 3칸 이동'
- (1, 3) 이 벽이라 1칸 가고 멈췄다
- 로그 : [벽에 막혀 이동 중단]
- 3회차(온디바이스 연동) : 실제 이동 결과를 서버로 보낸다



도전 문제 3 - 손으로 방향 주기

Confidential

위를 가리키는 손 사진(pointing_up.jpg)으로 '이쪽으로 2칸 이동' 을 보낸다.
도구 · 답변 · 이동 칸 수를 먼저 예측한 뒤, 실행해서 확인하시오.

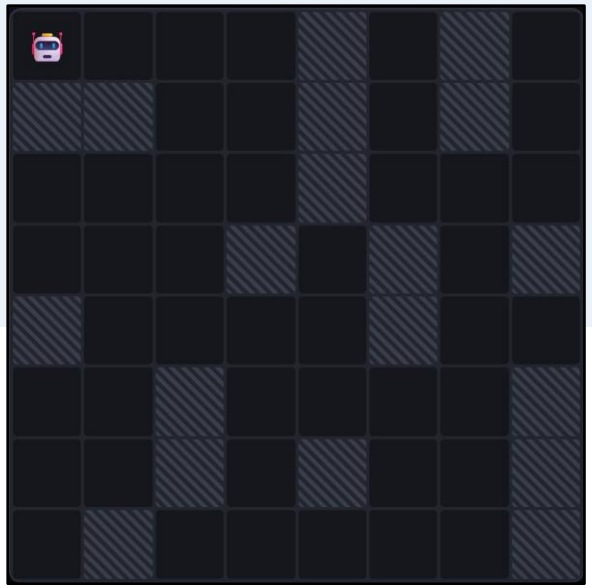
먼저 예측하세요

Q1. 어떤 도구와 인자가 나올까?

Q2. 답변창에는 뭐라고 나올까?

Q3. 로봇은 몇 칸 움직일까?

(로봇은 출발점 (1, 1) 에 있다)



힌트

· '이쪽' 은 방향어가 아니다

→ 그럼 R3 가 막을까?

· [손 제스처] 블록을 떠올려 보자

· (1, 1) 은 격자의 어디인가?

도전 3 해설 - 손이 방향을 정하고, LLM 이 move() tool을 선택했다

Confidential

move(up, 2) 는 맞았지만 로봇은 이동하지 않는다

- **A1. 손 방향 '위'(83%) → move(up, 2)**
- **R3 통과 - 손 방향이 근거가 된다**
- **A2. '위로 2칸 이동합니다'**
- **A3. 0칸 - 맨 윗줄이라 못 올라감**



Confidential

실습 중 막히면 여기부터 확인한다

증상	원인	해결
로봇이 안 움직인다	2일차 서버를 띄움 (tool 없음) / describe 선택	대시보드_완성.py 실행, 서버 창 [tool] 줄 확인
방향을 말했는데 되묻는다	방향어 정규식이 못 잡음	도전 2 – 방향어 목록에 그 표현을 더한다
'앞으로 2칸 가' 를 되묻는다	'앞' 은 격자 방향이 아니다	정상. 위 · 아래 · 왼쪽 · 오른쪽으로 말한다
몇 칸 가다가 멈춘다	벽 · 격자 경계	정상. 시뮬레이터 이동 경로 로그 확인
되묻기 뒤 칸 수가 1 로	R5 누락 – ask 의 args 에 칸 수 없음	pending_steps() 확인
API_검증.py 가 ConnectError	서버를 안 띄움	터미널 1 에서 서버 먼저
첫 요청만 오래 걸림	모델 지연 로딩	정상. 두 번째부터 빨라진다

Confidential

내용 정리

- ① 브라우저와 서버는 정해진 형식(API)으로만 데이터를 주고받는다 - 번호 하나로 설계 · 코드 · 검증이 연결된다
- ② 의도 파악 = 도구 고르기 - LLM 이 도구를 고르면, 그 도구대로 실행된다
- ③ 소형 LLM 은 모르면 추측한다 - 프롬프트 규칙으로는 막지 못했다 (6/8, 도전 1)
- ④ 조건 분기는 코드가 지킨다 - 근거 없는 move 는 ask, 되묻기 뒤 칸 수도 코드가 기억한다 (8/8)
- ⑤ 규칙을 고친 뒤에는 막을 것과 통과시킬 것을 둘 다 확인한다 - 방향어 목록 수정 (도전 2)
- ⑥ 완성 서버 - 웹캠 질문 · 손 제스처 → 도구 호출 → 시뮬레이터 로봇 이동 (43/43, 도전 3)

Confidential

필수 및 선택 기능 구현

- **① (필수) 새 도구를 설계해 본다**
  - 예 : stop, turn. API_설계에 번호를 붙여 적는다 (설계만) - 참고 : 11~15쪽 (API 설계)
- **② (필수) API_검증.py 에 내 질문 두 개를 항목으로 추가한다**
  - 참고 : 16~17쪽 (실습 1-1)
- **③ (선택) 내 손 제스처로 방향을 정해 본다**
  - 손으로 왼쪽 · 오른쪽을 가리키며 '이쪽으로 2칸 이동', 스트리밍 화면에서 좌우를 확인한다 - 참고 : 46~47쪽 (도전 3)
- **④ (선택) 손 제스처로만 프롬프트를 입력한다**
  - 프롬프트 입력 트리거 포즈를 정하고, 방향과 이동할 칸수 정보를 같이 인식하도록 처리한다

Confidential

오늘 만든 AI Workflow 를 온디바이스 환경에서 재구현한다

- **라즈베리파이에 카메라 · 마이크 모듈을 연결한다**
  - 같은 API 를 라즈베리파이가 호출한다 - 오늘의 설계 문서와 검증 스크립트를 그대로 쓴다
- **판단은 경량 모델로 다시 구현한다**
  - PC 방식과 온디바이스 방식의 속도 · 정확도 · 네트워크 의존성을 비교한다
- **도구가 시뮬레이터 대신 실제 장치를 제어한다**
  - move 결과를 LED · OLED 로 보여 주고, 실행 결과를 되돌려 받아 '답과 실제가 다른' 문제를 해결한다
- **지금은 비어 있는 대시보드의 '연동'탭 기능을 완성한다**

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Q & A

Confidential

모든 명령은 C:₩work₩core_pjt₩source₩2회차_day3 에서 실행한다

- 서버는 띄운 채로 둔다
- 검증은 다른 터미널에서 실행한다
- 대시보드는 브라우저로 연다
- 설계 문서는 API_설계

```
# 실습 한 건 = 폴더 한 개. 폴더 앞 숫자 = 이 강의자료 쪽번호
cd C:\work\core_pjt\source\2회차_day3

# Part 1 — API 설계
#   터미널 1 : 2일차 서버 (검증이 끝나면 Ctrl+C)
cd ..\2회차_day2\page64_대시보드_LLM연결 && python 대시보드_LLM연결.py
#   터미널 2 : 검증
cd C:\work\core_pjt\source\2회차_day3\page16_API_검증 && python API_검증.py

# Part 2 — 의도 파악과 도구 호출
cd ..\page21_도구선택     && python 도구선택.py

# Part 3 — 조건 분기
cd ..\page31_조건분기     && python 조건분기.py

# Part 4 — 대시보드 완성
#   터미널 1 : 완성 서버 → 브라우저 http://127.0.0.1:8000
cd ..\page39_대시보드_완성 && python 대시보드_완성.py
#   터미널 2 : 검증
cd C:\work\core_pjt\source\2회차_day3\page16_API_검증 && python API_검증.py
```

Confidential

API 번호로 설계 문서 · 서버 코드 · 검증 항목을 찾아간다

번호	설계 (API_설계)	서버 (대시보드_완성.py)	검증 (API_검증.py)
A1 · A2	화면 · 제스처 파일	index() · gesture_model() · mount	run_quick
A3 · A4	YOLO · VLM	yolo_detect() · vlm_describe()	run_quick · run_models
A5	도구 호출 응답	llm_answer() · decide_tool()	TOOL_CASES
R1 · R2	도구 · 인자 형식	parse_tool_call() · to_steps()	check_shape
R3	근거 없는 move → ask	apply_rules() · has_direction_evidence()	TOOL_CASES 의 R3
R4	need_more_info	llm_answer()	check_shape
R5	되묻기 뒤 방향만 답 → move	apply_rules() · pending_steps()	R5 2턴

Confidential

브라우저와 서버가 주고받는 형식 - API 설계 요약

- **오류 응답은 모두 {"error": "코드"} - 오늘 바뀌는 것은 A5 하나 (A1 ~ A4 는 2일차 서버와 같다)**

번호	메서드 · 경로	요청	응답 (200)	오류
A1	GET /	—	index.html	—
A2	GET /mediapipe/...	—	제스처 인식 파일 (vision_bundle.mjs · wasm · .task)	404
A3	POST /api/yolo-detect	JPEG 바이트	{objects: [{label, confidence, box}]}	400 empty_image · invalid_image, 500 yolo_failed
A4	POST /api/vlm-describe	JPEG 바이트	{description: 문자열}	400 empty_image · invalid_image, 500 vlm_failed
A5	POST /api/llm-answer	JSON (아래 표)	JSON (다음 쪽)	400 invalid_json · empty_prompt, 500 llm_failed

A5 요청 필드	필수	설명
prompt	O	사용자 질문. 공백뿐이면 400 empty_prompt
yolo_summary		A3 결과 요약 (예 : person 3개, bus 1개)
vlm_description		A4 결과 문장
gesture_direction		손 방향 up · down · left · right · unknown
history		되묻기(ask) 뒤 후속 턴일 때만 [{role, content}]

Confidential

A5 응답 = 도구 호출 - tool · args · need_more_info · message

- **규칙 R1 ~ R5 는 LLM 이 틀려도 코드가 보장한다 - 서버 주석의 [R1] ~ [R5] 와 같은 번호**

tool	의도 (예)	args	브라우저 동작
move	이동 (오른쪽으로 3칸 가)	{direction: up|down|left|right, steps: 1~10}	시뮬레이터 로봇 이동, 벽 · 경계에서 멈춤
describe	장면 질문 (앞에 뭐 있어?)	{}	답변만 표시, 이동 단계 건너뜀
ask	방향 모름 (저쪽으로 가)	{} 또는 {steps: 아는 칸 수}	[추가 정보 요청] 표시, 대화 기록 보관

번호	규칙	보장하는 곳
R1	tool 은 move · describe · ask 중 하나 (깨진 JSON · 없는 도구 → describe)	parse_tool_call
R2	인자 형식 — 방향은 네 개 중 하나, steps 1 ~ 10. 방향 없는 move → ask	parse_tool_call · to_steps
R3	방향 근거(질문의 방향어 · 이전 턴 · 손 제스처)가 없으면 move → ask	apply_rules · has_direction_evidence
R4	need_more_info == (tool == ask)	llm_answer
R5	ask 뒤 방향만 답하면 move — 칸 수는 ask 가 기억한 값	apply_rules · pending_steps

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

내일 방송에서 만나요!

삼성청년SW·AI아카데미